Retrieved but Ignored: Where LLMs Lose Code Reuse When Refactoring

Evgenii Lialin and Roman Vikhman
[TODO: affiliation]
[TODO: emails]

Abstract

When language models refactor code, they often re-implement logic that the repository already provides. We ask where such reuse opportunities are lost: in retrieving the existing helper, in the model's decision to call it, or in the edit itself. We build a benchmark of 129 test-validated reuse tasks from 12 Python repositories by inlining a helper into one of its callers, so that the original caller is a known reference. We then vary the context (none, BM25 snippets, the helper guaranteed among distractors, the whole module) and the instruction (generic vs. explicit reuse) for four open models. For three of the four models, the decision is the main bottleneck: even with the helper guaranteed in context and a generic instruction, they ignore it in 62–81% of tasks, while BM25 misses it in only 24%. An explicit reuse instruction raises reuse in every condition, by up to 2.8×. A newer coder model (Qwen3-Coder-30B-A3B) produces far more correct code than 7B models but does not reuse more; only the newest general model (Qwen3.8-27B) does. Reuse also depends on where the helper appears in the context and on its name: showing the helper under a neutral name lowers reuse substantially, while direct memorisation of the original code is rare.

1 Introduction

Code assistants increasingly edit existing repositories rather than write code from scratch. A well-known weakness is that they duplicate functionality the project already has: AI-generated pull requests contain more redundant code than human ones (Huang et al., 2026), and pretrained models frequently re-implement repository dependencies instead of calling them (Hai et al., 2025). Duplicated logic is a maintenance cost that tests do not reveal.

Refactoring benchmarks evaluate whether edits preserve behaviour or match human refactorings (Xu et al., 2026; Gautam et al., 2025; Thillen et al., 2026), and retrieval-augmented systems supply repository context for completion, repair and refactoring (Zhang et al., 2025; Xiong et al., 2025; Xu et al., 2025). None of them isolates *reuse*: whether a model, given a function that duplicates an existing helper, replaces the duplicate with a call. When reuse fails, it is also unclear why: the helper may not have been retrieved, the model may have seen it and ignored it, or the call may have been made incorrectly.

We study this with a controlled benchmark and a *reuse funnel* that assigns every failure to exactly one of these stages. Our contributions:

- A benchmark of 129 reuse-refactoring tasks in 12 Python repositories, built by inlining a helper into its caller and kept only when the call site is covered by tests that still pass (§3).
- An evaluation of four open models under four context conditions and two instructions, with failures attributed to retrieval, decision or edit (§5).
- Two controls that change the conclusions: the helper's position among retrieved snippets changes reuse for three of four models, and a renaming check separates naming cues from memorisation of the public repositories.

2 Related Work

Refactoring with LLMs. SWE-Refactor (Xu et al., 2026) and MANTRA (Xu et al., 2025) evaluate behaviour-preserving refactorings in Java; RefactorBench (Gautam et al., 2025) poses multi-file Python refactorings to agents; CodeTaste (Thillen et al., 2026) asks whether models choose human-like refactorings. They measure correctness or agreement with humans, not reuse of existing code.

Repository context and reuse. RepoExec (Hai et al., 2025) introduces the dependency invocation

rate for repository-level generation and finds that models often re-implement available dependencies; DependEval (Du et al., 2025) benchmarks dependency understanding. We adopt the idea of measuring invocation, but for refactoring, where the duplicated logic is already present and must be recognised.

Retrieval for code. CodeRAG (Zhang et al., 2025), ReflectCode (Liu et al., 2025), CoRet (Fehr et al., 2025) and Think-Search-Patch (Xiong et al., 2025) improve retrieval for completion, change requests, editing and repair. We use BM25 (Robertson and Zaragoza, 2009) and an oracle to separate retrieval quality from what the model does with the retrieved code.

Instructions and context position. CodeIF (Yan et al., 2025) shows that instruction following in code generation varies widely; we compare a generic refactoring instruction with an explicit reuse instruction. Models use information unevenly across long contexts (Liu et al., 2024); we test whether the helper's position among retrieved snippets matters.

3 Benchmark

Task construction. For each repository we find statement-level calls from a function F to a repository helper H (a module function, a method called via `self`, or a function of an imported module). We inline H's body at the call site, binding arguments, renaming locals to avoid clashes and turning early returns of H into returns of F. The mutated F duplicates H; the original F, which calls H, is the reference. A task is kept only if the call site is covered by the test suite and the full suite still passes after inlining. Inlining an overridable method would change subclass behaviour, so methods defined by more than one class are skipped.

Data. We use 12 Python libraries with fast test suites (boltons, cachetools, click, mistune, parso, pycparser, pyflakes, pygments, python-tabulate, sqlparse, tomlkit, toolz) at pinned commits. We keep every task whose helper lives in another module (36 tasks) and a seeded sample of same-module tasks, up to 12 per repository, giving 129 tasks. Helpers have 2–30 lines (median 7); 64 are methods called via `self`, 61 plain functions and 4 module attributes.

Conditions. All conditions except FILE share a base context: the module's imports and the mutated F (with its class header). NONE adds nothing. BM25 adds the top-5 repository functions retrieved with BM25 using F as the query; the original F is excluded from the index. ORACLE adds H and the top-4 BM25 distractors, with H at a seeded random position; ORACLE-FIRST/LAST place it first or last (§5.1). FILE shows the whole mutated module, truncated to 30k characters around F (53 tasks); it contains H only for same-module helpers (80 tasks after truncation). Each condition is crossed with two instructions: *generic* ("Refactor this function to improve its readability and maintainability") and *reuse*, which adds "Where possible, reuse existing functions from the project instead of reimplementing their logic." Both ask the model to keep the name, signature and behaviour and to output only the changed function and any new helpers.

Evaluation. The model's function replaces F; new helper functions or methods are added, but redefinitions of existing code are dropped and the output is marked non-compliant. We run the repository's tests in a sandbox. An output is *valid* if it is compliant and the tests pass; it *reuses* the helper if the new F calls H or a near-identical copy of H elsewhere in the repository, resolved through imports. The *reuse funnel* assigns each task to one stage: H *not in context*, in context but *ignored*, called but *broken* (tests fail), or success. Conditions and models are compared per task with exact McNemar tests.

4 Experimental Setup

We evaluate Qwen2.5-Coder-7B-Instruct (Q2.5C-7B; Hui et al., 2024) and DeepSeek-Coder-6.7B-Instruct (DSC-6.7B; Guo et al., 2024) in bf16 with Hugging Face Transformers, and two 2025–26 models in 4-bit quantisation with vLLM (Kwon et al., 2023): Qwen3-Coder-30B-A3B-Instruct (Q3C-30B, a mixture of experts with 3B active parameters; Yang et al., 2025; Qwen Team, 2025) and Qwen3.8-27B (Q3.8-27B, thinking disabled; Qwen Team, 2026). All runs use greedy decoding, at most 2,048 new tokens, and a single 24 GB GPU; each model answers 1,032 prompts (129 tasks $\times$ 4 conditions $\times$ 2 instructions), plus 516 for the position ablation.

Model	Instr.	NONE	BM25	ORACLE	FILE
Q2.5C-7B	generic	1 (49)	21 (60)	38 (59)	18 (52)
	reuse	3 (43)	43 (56)	53 (53)	22 (52)
DSC-6.7B	generic	4 (36)	33 (45)	36 (42)	18 (47)
	reuse	5 (37)	47 (45)	45 (42)	19 (43)
Q3C-30B	generic	2 (91)	22 (91)	19 (88)	9 (93)
	reuse	5 (90)	41 (87)	54 (83)	27 (88)
Q3.8-27B	generic	5 (90)	50 (92)	53 (88)	36 (96)
	reuse	12 (92)	74 (92)	84 (88)	64 (93)

Table 1: Helper reuse in % of 129 tasks (valid outputs in % in parentheses).

	BM25		ORACLE	
Model	generic	reuse	generic	reuse
Q2.5C-7B	31/73/4/21	31/49/9/40	0/80/14/35	0/60/17/52
DSC-6.7B	31/60/13/25	31/41/21/36	0/83/19/27	0/71/21/37
Q3C-30B	31/73/0/25	31/50/4/44	0/105/0/24	0/59/4/66
Q3.8-27B	31/40/1/57	31/13/3/82	0/60/5/64	0/21/3/105

Table 2: Reuse funnel over 129 tasks: not in context / ignored / broken / success.

5 Results

Without context, models re-implement. In NONE, reuse is 1–5% for three models and at most 12% for Q3.8-27B (Table 1). Instead, the smaller models write a new helper of their own in 68% (Q2.5C-7B) and 48% (DSC-6.7B) of outputs (Q3C-30B 15%, Q3.8-27B 10%). Many of the remaining calls without context are explained by the prompt itself: the helper's name already appears in the module's imports in 44 of the 258 NONE prompts.

The decision, not retrieval, is the main loss. BM25 places the helper in the top 5 for 76% of tasks (MRR 0.58), so retrieval loses 31 tasks. With the helper guaranteed in context and the generic instruction, three models ignore it in 80–105 of 129 tasks, and even Q3.8-27B in 60 (Table 2). Calls that break the tests are rare for the newer models (0–5) but not for the 7B models (up to 21). Only for Q3.8-27B with the reuse instruction does retrieval become the larger loss (31 not retrieved vs. 13 ignored). Whole-module context yields the least reuse of the three context conditions for every model, although the helper is visible in 80 tasks.

An explicit instruction raises reuse. Adding the reuse sentence raises reuse in every condition with context, e.g. ORACLE from 19% to 54% for Q3C-30B and from 53% to 84% for Q3.8-27B, while validity drops by at most 6 points. Models can reuse, but they do not consider it part of refactoring unless asked.

Stronger is not the same as more reusing. Q3C-30B produces valid code in 83–93% of cases versus 36–60% for the 7B models, yet it does not reuse significantly more than Q2.5C-7B in any condition; with ORACLE and the generic instruction it reuses less (10 vs. 35 discordant tasks, $p < .001$). Q3.8-27B reuses more than both in every condition with context ($p \leq .002$), e.g. BM25 with the reuse instruction 74% vs. 41–47%.

5.1 Position of the helper

DSC-6.7B reused the helper more with BM25 than with ORACLE, although ORACLE always contains it. BM25 usually ranks the helper first, so we repeated ORACLE with the helper first or last, with identical snippets otherwise (Table 3). Three models show a primacy effect, but under different instructions: DSC-6.7B only with the reuse instruction (75 vs. 55 tasks, $p < .001$), Q3C-30B and Q3.8-27B only with the generic one (37 vs. 25 and 78 vs. 69). Q2.5C-7B shows none. Reordering identical snippets changes the outcome of up to 30 tasks (23%) per model and instruction. Oracle upper bounds should therefore fix the helper's position, and differences between conditions should be tested per task rather than read from aggregate rates.

5.2 Memorisation or naming?

All repositories are public and predate the models, so a model might reuse the helper because it remembers the original caller. We re-ran the two newer models with the helper shown under a neutral name (`fn_<hash>`) everywhere in the prompt; outputs are mapped back before testing, and any use of the original, never-shown name is counted as memorised and broken. We use the 114 tasks whose helper name is unique and not also a builtin or standard-library name. Memorisation is rare: 18 of 912 outputs for Q3.8-27B and 5 for Q3C-30B use the original name, almost all for private sibling methods such as `self._parse_key`. Without context, reuse falls to 2 of 114 for both models. Neutral names, however, reduce reuse substantially for both models (Table 4): with BM25 and the generic instruction both lose about 60% of their reuse. Since the control model loses a similar share, the drop reflects the loss of a descriptive name as a cue rather than memorisation specific to Q3.8-27B. Q3.8-27B

Model	generic first	random	last	first/last only	p	reuse first	random	last	first/last only	p
Q2.5C-7B	40	49	41	13/14	1.0	63	69	67	12/16	.572
DSC-6.7B	47	46	47	11/11	1.0	75	58	55	25/5	<.001
Q3C-30B	37	24	25	14/2	.004	70	70	66	17/13	.585
Q3.8-27B	78	69	69	12/3	.035	110	108	108	4/2	.688

Table 3: Tasks (of 129) where the helper is reused when placed first, at a seeded random position, or last among the five ORACLE snippets; discordant tasks and exact McNemar p for first vs. last.

Model	Instr.	BM25	ORACLE	FILE
Q3C-30B	generic	23→9 (<.001)	19→12 (.065)	11→8 (.508)
	reuse	46→36 (.031)	59→46 (.004)	29→22 (.143)
Q3.8-27B	generic	56→22 (<.001)	61→33 (<.001)	39→21 (<.001)
	reuse	83→62 (<.001)	96→82 (<.001)	70→59 (.003)

Table 4: Helper reused (of 114 tasks) with the original name → with a neutral name in the prompt (McNemar p).

still reuses far more than Q3C-30B under renaming (ORACLE, reuse instruction: 37 vs. 1 discordant tasks, $p < .001$).

6 Conclusion

Reuse in LLM refactoring is lost mainly at the model's decision, not at retrieval: models see the helper and ignore it, unless explicitly asked to reuse. Better retrieval alone would therefore recover few tasks for most models. Higher code quality does not imply more reuse, and reuse depends on surface cues such as the helper's position and name. Benchmarks of repository-aware code editing should report reuse alongside test results, fix the position of oracle context and control for naming. We release the benchmark, prompts, outputs and evaluation code.[TODO: repository link]

Limitations

Tasks are created by deterministic inlining, so the duplicated code is a near-verbatim copy of the helper; real duplication is usually paraphrased, which would make reuse harder to recognise, and a set of tasks mined from real commits is future work. We use 12 popular Python libraries; results may differ for other languages and for private code. Passing tests do not prove equivalence, although we only keep tasks whose call site is covered. We evaluate four open models with greedy decoding and a single run, two of them quantised to 4 bits; we do not evaluate proprietary models, dense or agentic retrieval. Our renaming check changes names only in the prompt; it does not rule out memorisation of code structure. A small number of outputs were re-evaluated on a different machine after harness fixes, where test outcomes can differ slightly.

References

Junjia Du, Yadi Liu, Hongcheng Guo, Jiawei Wang, Haojian Huang, Yunyi Ni, and Zhoujun Li. 2025. DependEval: Benchmarking LLMs for repository dependency understanding. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 7150–7179, Vienna, Austria. Association for Computational Linguistics.

Fabio James Fehr, Prabhu Teja S, Luca Franceschi, and Giovanni Zappella. 2025. CoRet: Improved retriever for code editing. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 775–789, Vienna, Austria. Association for Computational Linguistics.

Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. 2025. Refactorbench: Evaluating stateful reasoning in language agents through code. *Preprint*, arXiv:2503.07832. Published at ICLR 2025.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *Preprint*, arXiv:2401.14196.

Nam Le Hai, Dung Manh Nguyen, and Nghi D. Q. Bui. 2025. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1496–1524, Albuquerque, New Mexico. Association for Computational Linguistics.

Haoming Huang, Pongchai Jaisri, Shota Shimizu, Lingfeng Chen, Sota Nakashima, and Gema Rodríguez-Pérez. 2026. More code, less reuse: Investigating code quality and reviewer sentiment towards ai-generated pull requests. *Preprint*, arXiv:2601.21276.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, and 5 others. 2024. Qwen2.5-coder technical report. *Preprint*, arXiv:2409.12186.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. *Preprint*, arXiv:2309.06180.

Aofan Liu, Song Shiyuan, Haoxuan Li, Cehao Yang, and Yiyan Qi. 2025. Beyond function-level search: Repository-aware dual-encoder code retrieval with adversarial verification. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 21034–21049, Suzhou, China. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Qwen Team. 2025. Qwen3-Coder-30B-A3B-Instruct-AWQ-4bit. `https://huggingface.co/cyankiwi/Qwen3-Coder-30B-A3B-Instruct-AWQ-4bit`. 4-bit AWQ quantization of Qwen3-Coder-30B-A3B-Instruct.

Qwen Team. 2026. Qwen3.8-27B-INT4. `https://huggingface.co/RedHatAI/Qwen3.8-27B-INT4`. INT4 quantization of Qwen3.8-27B.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Alex Thillen, Niels Mündler, Veselin Raychev, and Martin Vechev. 2026. Codetaste: Can llms generate human-level code refactorings? *Preprint*, arXiv:2603.04177. Published at ICML 2026.

Bojian Xiong, Yikun Lei, Xikai Liu, Shaowei Zhang, Pengyun Zhu, Yan Liu, Yongqi Leng, Ling Shi, Meizhi Zhong, Yurong Zhang, Yan Gao, Yiwu, Yao Hu, and Deyi Xiong. 2025. Think-search-patch: A retrieval-augmented reasoning framework for repository-level code repair. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1555–1566, Suzhou (China). Association for Computational Linguistics.

Yisen Xu, Feng Lin, Jinqiu Yang, Tse-Hsun Chen, and Nikolaos Tsantalis. 2025. Mantra: Enhancing automated method-level refactoring with contextual rag and multi-agent llm collaboration. *Preprint*, arXiv:2503.14340.

Yisen Xu, Jinqiu Yang, and Tse-Hsun Chen. 2026. Swe-refactor: A repository-level benchmark for real-world llm-based code refactoring. *Preprint*, arXiv:2602.03712.

Kaiwen Yan, Hongcheng Guo, Xuanqing Shi, Shaosheng Cao, Donglin Di, and Zhoujun Li. 2025. CodeIF: Benchmarking the instruction-following capabilities of large language models for code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 1272–1286, Vienna, Austria. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Sheng Zhang, Yifan Ding, Shuquan Lian, Shun Song, and Hui Li. 2025. CodeRAG: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 23278–23288, Suzhou, China. Association for Computational Linguistics.